

Simulation - Network-based

2026-04-30

Define the population $\mathcal{I}$ and interference structure M .

For the network example, we use a network-derived interference structure from the Transmission Reduction Intervention Project (TRIP). The data provide an observed ego-alter edge list, where `EGO_ID` identifies the focal individual and `ALTER_ID` identifies that individual's network contact.

We use the observed ego-alter pairs to construct the network with `build_network()`. Interference is assumed to occur through immediate observed network neighbors.

For individual i , the interference set $\mathcal{N}_i$ consists of the individuals connected to i in the observed TRIP network. Thus, $|\mathcal{N}_i|$ is the observed number of network neighbors of individual i .

After generating the edge list, we use the package function `build_network()` to create the network object. We also independently generate a binary individual-level covariate

$$L_i \sim \text{Bernoulli}(0.4), \quad i \in \mathcal{I}.$$

```
set.seed(999)

load("/Users/dingyudupan/Desktop/39059-0001-Data.rda")

df <- da39059.0001

net.network <- build_network(
  x1 = df$EGO_ID,
  x2 = df$ALTER_ID
)

# create network_id from observed connected components
net.network$data$network_id <- igraph::components(net.network$net0)$membership[
  net.network$data$id
]

net.network$data$L_i <- rbinom(nrow(net.network$data), 1, 0.4)
```

Simulate $Y(a_i, a_{\mathcal{N}_i})$.

The outcome-generating mechanism corresponds to a binary outcome and is specified using the aggregated network-neighbor exposure mapping $\bar{a}_{\mathcal{N}_i}$:

$$Y(a_i, a_{\mathcal{N}_i}) = Y(a_i, \bar{a}_{\mathcal{N}_i}) = \text{logit}^{-1}(0.5 - 0.5a_i - 0.5\bar{a}_{\mathcal{N}_i} + 0.25a_i\bar{a}_{\mathcal{N}_i} + 0.25L_i).$$

Coefficients are chosen to induce both direct and spillover effects and to allow for interaction between individual and peer exposure; their specific values are not substantively interpreted.

Simulate A_i and $\bar{A}_{\mathcal{N}_i}$.

Given the treatment assignment mechanism, we generate a single realization of the treatment vector $\mathbf{A} = (A_1, \dots, A_n)$. For each individual i , the aggregated exposure $\bar{A}_{\mathcal{N}_i}$ is defined as the mean treatment status among individuals in i 's network-neighbor interference set, excluding i themself.

We assume that treatment is assigned according to the following individual-level model:

$$A_i \sim \text{Bernoulli}\{\text{logit}^{-1}(-0.5 + L_i)\}.$$

```
sim.network <- simulate_network_outcome(
  net.network,
  outcome_intercept = 0.5,
  beta_a = -0.5,
  beta_abar = -0.5,
  beta_a_abar = 0.25,
  outcome_covars = "L_i",
  outcome_coefs = 0.25,
  propensity_intercept = -0.5,
  treatment_covars = "L_i",
  treatment_coefs = 1
)

PO <- sim.network[[1]]
dat.network <- sim.network[[2]]
```

`PO` is a list of matrices, with each matrix corresponding to an individual i and encoding that individual's potential outcomes under different treatment and interference scenarios. Within each matrix, the rows index the individual's own treatment status, with the first row representing individual i not treated ($A_i = 0$) and the second row representing individual i treated ($A_i = 1$). The columns index the level of exposure to interference, where column k corresponds to the case in which exactly k individuals in i 's network-neighbor interference set are treated, for $k = 0, 1, \dots, N_i$, with N_i denoting the size of the interference set for individual i .

`dat.network` is observed outcome using consistency $Y_{\text{obs},i} = PO_i[A_i + 1, A_{\mathcal{N}_i} + 1]$.

```
PO[[1]]
```

```
##      [,1] [,2]
## [1,]    0    1
## [2,]    0    0
```

```
PO[[2]]
```

```
##      [,1] [,2]
## [1,]    0    1
## [2,]    1    0
```

```
head(dat.network)
```

```
##  subject_id id network_id L_i interference_set_size Ai treated_neighbors
## 1      2021  1          1    0                      1    0                1
## 2      2028  2          1    0                      1    0                1
```

```
## 3      2032 3      1 0      5 1      3
## 4      2036 4      1 1     10 0      6
## 5      2038 5      1 1     10 0      4
## 6      2061 6      1 0      2 0      2
##  Abar_Ni Y_obs
## 1      1.0 1
## 2      1.0 1
## 3      0.6 1
## 4      0.6 0
## 5      0.4 1
## 6      1.0 1
```

Calculate true causal effects

The object `PO` stores individual-level potential outcomes under network interference. For each individual i , `PO[[i]]` is a $2 \times (N_i + 1)$ matrix, where rows correspond to the individual's own treatment status ($A_i = 0$ or $A_i = 1$), and columns correspond to the number of treated individuals in i 's network-neighbor interference set ($k = 0, \dots, N_i$).

To compute true causal estimands under a Bernoulli allocation policy with treatment probability α , we average potential outcomes over the distribution of the number of treated individuals in the network-neighbor interference set, induced by independently assigning treatment to each neighbor with probability α . For fixed (a, α) , the individual-level average potential outcome is defined as

$$\bar{Y}_i(a_i = a, \alpha) = \sum_{k=0}^{|\mathcal{N}_i|} \text{PO}_i[a + 1, k] \binom{|\mathcal{N}_i|}{k} \alpha^k (1 - \alpha)^{|\mathcal{N}_i| - k}.$$

The package function below implements these quantities directly from `sim.network`.

```
true_val <- simulate_network_true_effect(
  sim.network,
  allocations = c(0.25, 0.5, 0.75)
)

true_val
```

```
## $allocations
## [1] 0.25 0.50 0.75
##
## $DE
##   effect alpha true_effect
## 1    DE  0.25  -0.1183112
## 2    DE  0.50  -0.1267048
## 3    DE  0.75  -0.1131035
##
## $IE
##   effect alpha1 alpha0 true_effect
## 1    IE  0.25   0.50  0.01831847
## 2    IE  0.25   0.75  0.04576748
## 3    IE  0.50   0.75  0.02744901
##
## $TE
```

```
##   effect alpha1 alpha0 true_effect
## 1    TE    0.25   0.50 -0.09999274
## 2    TE    0.25   0.75 -0.07254373
## 3    TE    0.50   0.75 -0.09925578
##
## $OE
##   effect alpha1 alpha0 true_effect
## 1    OE    0.25   0.50  0.05209306
## 2    OE    0.25   0.75  0.10101727
## 3    OE    0.50   0.75  0.04892421
##
## attr(,"class")
## [1] "network_true_effect"
```

Estimate the spillover effect using inference.

The group variable in `inference()` is `network_id`, which identifies the disconnected network component. There are no edges between different `network_id` groups.

```
model <- inference(
  Y_obs | Ai ~ L_i | network_id,
  allocations = c(0.25, 0.5),
  data = dat.network,
  model_method = "glm",
  model_options = list(family = binomial(link = "logit"))
)
```

```
## Warning: the 'findbars' function has moved to the reformulas package. Please update your imports, or
## This warning is displayed once per session.
```

```
model$estimates
```

	alpha1	trt1	alpha2	trt2	marginal	effect_type	effect	estimate	std.error
## 1	0.25	0	NA	NA	FALSE	outcome	outcome	0.10258287	0.05293118
## 2	0.50	0	NA	NA	FALSE	outcome	outcome	0.26060862	0.10329272
## 3	0.25	1	NA	NA	FALSE	outcome	outcome	0.15227387	0.09148407
## 4	0.50	1	NA	NA	FALSE	outcome	outcome	0.23614387	0.08203451
## 5	0.25	NA	NA	NA	TRUE	outcome	outcome	0.11500562	0.05349551
## 6	0.50	NA	NA	NA	TRUE	outcome	outcome	0.24837625	0.07394771
## 7	0.25	1	0.25	0	FALSE	contrast	direct	0.04969100	0.08428038
## 8	0.50	1	0.50	0	FALSE	contrast	direct	-0.02446475	0.11368834
## 9	0.25	0	0.25	1	FALSE	contrast	direct	-0.04969100	0.08428038
## 10	0.50	0	0.50	1	FALSE	contrast	direct	0.02446475	0.11368834
## 11	0.25	0	0.25	0	FALSE	contrast	indirect	0.00000000	0.00000000
## 12	0.50	0	0.25	0	FALSE	contrast	indirect	0.15802575	0.05405450
## 13	0.25	1	0.25	1	FALSE	contrast	indirect	0.00000000	0.00000000
## 14	0.50	1	0.25	1	FALSE	contrast	indirect	0.08387000	0.05740973
## 15	0.25	0	0.50	0	FALSE	contrast	indirect	-0.15802575	0.05405450
## 16	0.50	0	0.50	0	FALSE	contrast	indirect	0.00000000	0.00000000
## 17	0.25	1	0.50	1	FALSE	contrast	indirect	-0.08387000	0.05740973
## 18	0.50	1	0.50	1	FALSE	contrast	indirect	0.00000000	0.00000000
## 19	0.25	1	0.25	0	FALSE	contrast	total	0.04969100	0.08428038

```

## 20  0.50    1  0.25    0  FALSE  contrast  total  0.13356100 0.09163047
## 21  0.25    1  0.50    0  FALSE  contrast  total -0.10833475 0.10566575
## 22  0.50    1  0.50    0  FALSE  contrast  total -0.02446475 0.11368834
## 23  0.25    0  0.25    1  FALSE  contrast  total -0.04969100 0.08428038
## 24  0.50    0  0.25    1  FALSE  contrast  total  0.10833475 0.10566575
## 25  0.25    0  0.50    1  FALSE  contrast  total -0.13356100 0.09163047
## 26  0.50    0  0.50    1  FALSE  contrast  total  0.02446475 0.11368834
## 27  0.25   NA  0.25   NA   TRUE  contrast  overall 0.00000000 0.00000000
## 28  0.50   NA  0.25   NA   TRUE  contrast  overall 0.13337062 0.03858493
## 29  0.25   NA  0.50   NA   TRUE  contrast  overall -0.13337062 0.03858493
## 30  0.50   NA  0.50   NA   TRUE  contrast  overall 0.00000000 0.00000000
##      conf.low  conf.high
## 1 -0.001160331 0.20632608
## 2  0.058158614 0.46305863
## 3 -0.027031600 0.33157935
## 4  0.075359189 0.39692855
## 5  0.010156359 0.21985489
## 6  0.103441404 0.39331109
## 7 -0.115495510 0.21487751
## 8 -0.247289804 0.19836031
## 9 -0.214877510 0.11549551
## 10 -0.198360305 0.24728980
## 11 0.000000000 0.00000000
## 12 0.052080869 0.26397063
## 13 0.000000000 0.00000000
## 14 -0.028650999 0.19639099
## 15 -0.263970626 -0.05208087
## 16 0.000000000 0.00000000
## 17 -0.196390994 0.02865100
## 18 0.000000000 0.00000000
## 19 -0.115495510 0.21487751
## 20 -0.046031425 0.31315342
## 21 -0.315435812 0.09876632
## 22 -0.247289804 0.19836031
## 23 -0.214877510 0.11549551
## 24 -0.098766317 0.31543581
## 25 -0.313153421 0.04603143
## 26 -0.198360305 0.24728980
## 27 0.000000000 0.00000000
## 28 0.057745554 0.20899569
## 29 -0.208995691 -0.05774555
## 30 0.000000000 0.00000000

```

Simulation framework to evaluate and compare true versus estimated spillover (indirect) effects.

We conduct a simulation study to evaluate the performance of estimators for spillover effects under network interference. In each of 500 iterations, we use a fixed random network structure, regenerate the covariate, treatment, potential outcomes, and observed outcome, compute the true indirect effect under two allocation strategies (0.25 vs. 0.5) using the known data-generating mechanism, and estimate the corresponding indirect effect using `inference()`.

```

set.seed(999)

# load the dataset
load("/Users/dingyudupan/Desktop/39059-0001-Data.rda")

# define the dataset
df <- da39059.0001

# use observed ego-alter pairs from TRIP
network_edges <- data.frame(
  x1 = df$EGO_ID,
  x2 = df$ALTER_ID
)

# build observed TRIP network
interference.network <- build_network(
  x1 = network_edges$x1,
  x2 = network_edges$x2
)

# define network_id using connected components of the built network
interference.network$data$network_id <- igraph::components(interference.network$net0)$membership[
  interference.network$data$id
]

iter <- 500

sim.results <- data.frame(
  true_val = numeric(iter),
  estimate = numeric(iter),
  std.error = numeric(iter),
  conf.low = numeric(iter),
  conf.high = numeric(iter)
)

for (i in 1:iter) {
  net.i <- interference.network

  # generate individual-level covariate
  net.i$data$L_i <- rbinom(nrow(net.i$data), 1, 0.4)

  sim.network <- simulate_network_outcome(
    net.i,
    outcome_intercept = 0.5,
    beta_a = -0.5,
    beta_abar = -0.5,
    beta_a_abar = 0.25,
    outcome_covars = "L_i",
    outcome_coefs = 0.25,
    propensity_intercept = -0.5,
    treatment_covars = "L_i",
    treatment_coefs = 1
  )
}

```

```

dat.network <- sim.network[[2]]

true_val <- simulate_network_true_effect(
  sim.network,
  allocations = c(0.25, 0.5)
)

model <- interference(
  Y_obs | Ai ~ L_i | network_id,
  allocations = c(0.25, 0.5),
  data = dat.network,
  model_method = "glm",
  model_options = list(family = binomial(link = "logit"))
)

estimate_val <- subset(
  model$estimates,
  alpha1 == 0.25 & alpha2 == 0.5 &
  effect == "indirect" & trt1 == 0 & trt2 == 0
)

true_ie <- subset(
  true_val$IE,
  alpha1 == 0.25 & alpha0 == 0.5
)$true_effect

sim.results[i, ] <- c(
  true_ie,
  estimate_val$estimate,
  estimate_val$std.error,
  estimate_val$conf.low,
  estimate_val$conf.high
)
}

avg.bias <- mean(sim.results$true_val - sim.results$estimate)
ECP <- mean(between(sim.results$true_val, sim.results$conf.low, sim.results$conf.high))

avg.bias

## [1] 0.0001907362

ECP

## [1] 0.724

```